

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Technician Dispatch Agent

An AI Agent that assigns the Right Repair Technician to the Right Job.

Team _ | Batch A

AM.SC.U4AIE25008 · AM.SC.U4AIE25050 · AM.SC.U4AIE25035

Problem Statement

- Your car breaks down. Someone needs to decide: which mechanic to send, what route they take, and in what order the repair happens.
- We built a dispatch system that works all of that out automatically.
- Everything here is classical AI — no machine learning, no training data. Each decision is made by a distinct, well-known AI technique.
- Goal: show these techniques working together on one real pipeline, not in isolation.

Environment

- The city is a graph: junctions are nodes, roads are weighted edges (weight = distance).
- Repair shops and parts stores sit at fixed nodes on that graph.
- Each shop has a specialty (engine, brakes, etc.) and an availability flag.
- Each store has a stock list (can supply parts for several specialties at once) and availability.
- A breakdown is just a point on a road — snapped to the nearer junction to become the actual routing goal.

Environment

Fully observable — the whole map and every shop/store is known upfront.

Deterministic — same input always produces the same output.

Discrete — fixed set of nodes, edges, and entities.

Static — nothing on the map changes while the agent is reasoning about one request.

Single-agent — one dispatcher making the call, no competing agents in the environment itself.

PEAS Description

Performance measure: correct specialist match, shortest valid route, repair never starts before everything it needs has arrived.

Environment: the city graph, the shops/stores on it, the current breakdown.

Actuators: dispatch a technician, route a part, advance the repair plan.

Sensors: road clicked, exact position on it, damage type selected, answer to one follow-up question, whether the part is already on hand.

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Percepts

One percept per request: location, damage type, one follow-up answer, part-on-hand flag.

Single-shot perception — the agent doesn't watch the world continuously, it reasons once per dispatch.

That raw percept is converted into structured facts (specialist needed, urgency, parts needed) before anything else happens.

Those facts are what every later stage — CSP, search, planning — actually acts on.

Actions and Decisions

- What's actually wrong ?
- Which shop/store qualifies — right specialty/stock, available, close enough?
- Of the qualifying shops, which one gives the cheapest real route?
- If two jobs are competing, which gets served first?
- Is the repair step actually ready to start yet?
- How should technician and parts timing be synced?

Agent Architecture

Infer the facts from the percept.

Filter and select a shop (and a store, if parts are needed).

Compute and commit to a route.

Resolve the serve-first conflict.

Generate the ordered repair plan.

Coordinate technician/parts ETA, and replan if the technician is delayed.

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Methodology

1. Load the city map and shop/store data once, at server startup.
2. Turn the user's input into facts.
3. Filter candidate shops/stores against the constraints.
4. Route to every valid shop candidate, keep the cheapest.
5. Resolve serve-first priority.
6. Build the ordered repair plan.
7. Sync technician and parts arrival times.
8. Return the full reasoning trace + result to the frontend to animate.

Methodology: Knowledge

01 A plain rule base: one `when → then` rule per damage type (e.g. engine → specialist "engine", urgency 5, parts needed).

02 The inference engine applies these rules, then chains a second rule off the result it just derived.

03 A final override: if the user already has the part, `parts_needed` flips to false regardless of what the rule said.

04 This is forward chaining: start from facts, apply rules, derive new facts from those derived facts.

Methodology: Constraint Satisfaction

01 Each shop/store is checked against independent constraints: specialty/stock match, availability, maximum distance.

02 `allocate_shop` matches shops by exact specialty; `allocate_store` matches stores by stock list, since one store can carry parts for several specialties.

03 The result is the full list of valid candidates — a constraint filter, not a search.

04 Shops are then ranked by actual route cost; the cheapest valid one wins.

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Methodology: Search Algorithms

All three search the same weighted graph for a path from technician/store to the breakdown.

BFS

explores by depth, guarantees the fewest hops.

DFS

explores depth-first, no optimality guarantee, just a full path.

A*

uses real travel cost so far + straight-line distance to the goal as a heuristic, guaranteeing the optimal route.

All three run on every request so their performance can be compared side by side; A*'s route is the one actually used for ETA and the drawn path.

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Methodology: Adversarial Search

01 Models a simple question: if two breakdown requests compete, which gets dispatched first?

02 Framed as a 2-ply minimax: the dispatcher (MAX) picks who to serve; an adversarial layer (MIN) assumes the worst-case urgency escalation for whichever request is left waiting.

03 Alpha-beta pruning cuts off a branch once it can't beat the best option already found — shown here as `nodes_pruned`.

04 Same core technique used for move selection in games like chess, repurposed for a scheduling decision.

Methodology: Planning

The repair is broken into a fixed ordered sequence: diagnose → assign shop → assign store (if needed) → travel → repair → verify → close.

01 Each action has preconditions, and a step only counts as "ready" once they're satisfied.

02 The one step with a real precondition is "repair" — it needs both the technician to have arrived and the part to be either present or unnecessary.

03 Steps that aren't relevant to the current job (e.g. "assign store" when no part is needed) are skipped entirely.

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Methodology: Simulate Delay

Technician and parts travel independently and can arrive at different times
repair can't start until both are present.

The coordinator works out the gap between their ETAs and who's effectively waiting on whom.

The “Simulate delay” button applies a fixed 10-minute technician delay. It assumes “diagnose” and “assign shop” are complete, removes those steps from the plan, and recalculates the ETAs. This is a demonstration, not live progress tracking.

Demonstrated live via a "Simulate delay" action on top of the original plan.

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Performance Measures

1. Route comparison — BFS, DFS, and A* run on the same graph. The current test checks A* against BFS for one adjacent-node case.

2. Map structure — A test checks that the graph is connected and contains alternate routes and a dead end.

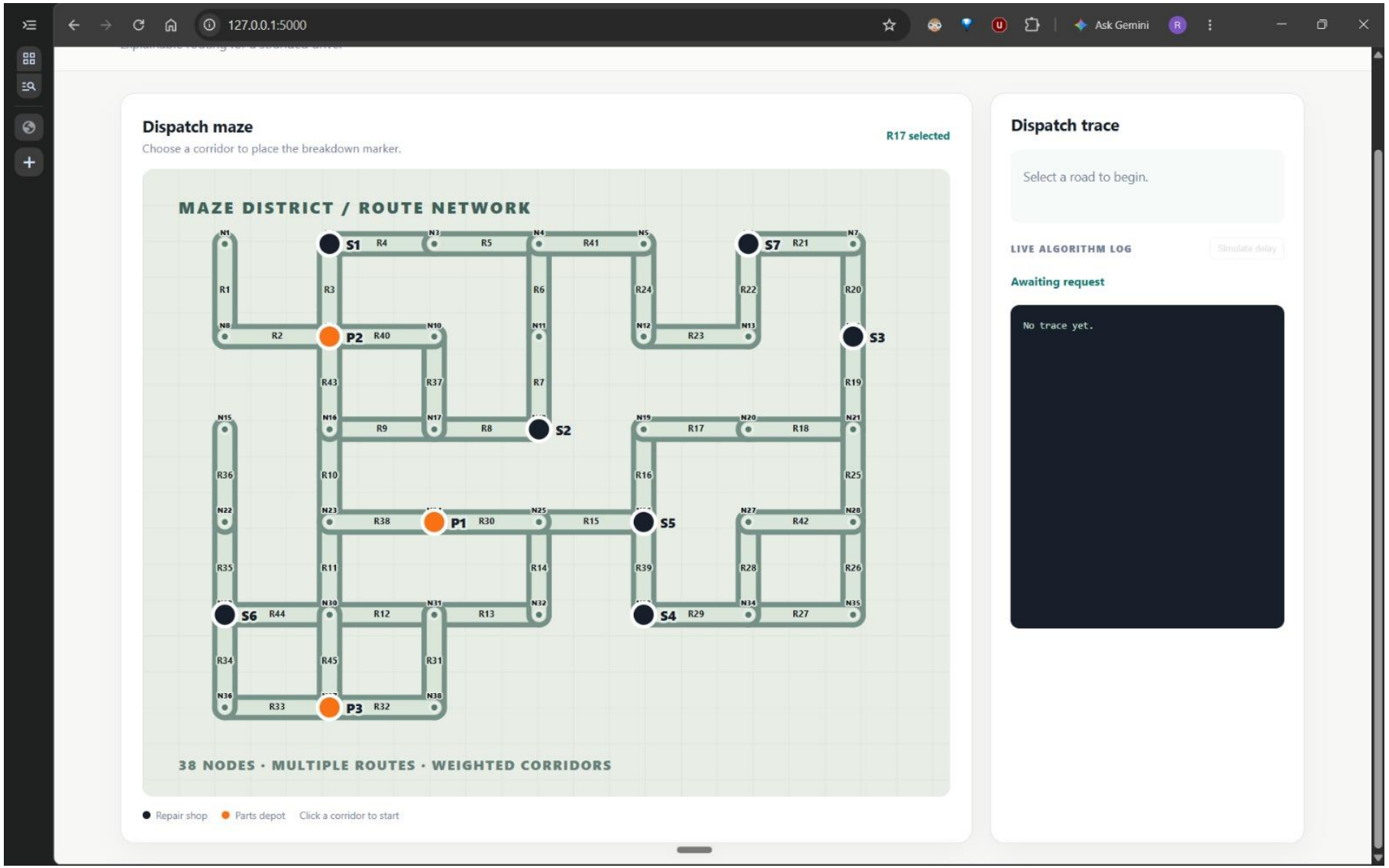
3. Shop specialty — A test checks that returned shop candidates match the requested specialty.

4. Repair readiness — Tests check that repair is not ready when neither the technician nor the part has arrived, and is ready when both have arrived.

5. Dispatch response — An endpoint test checks that dispatch returns a successful response containing a route and logs.

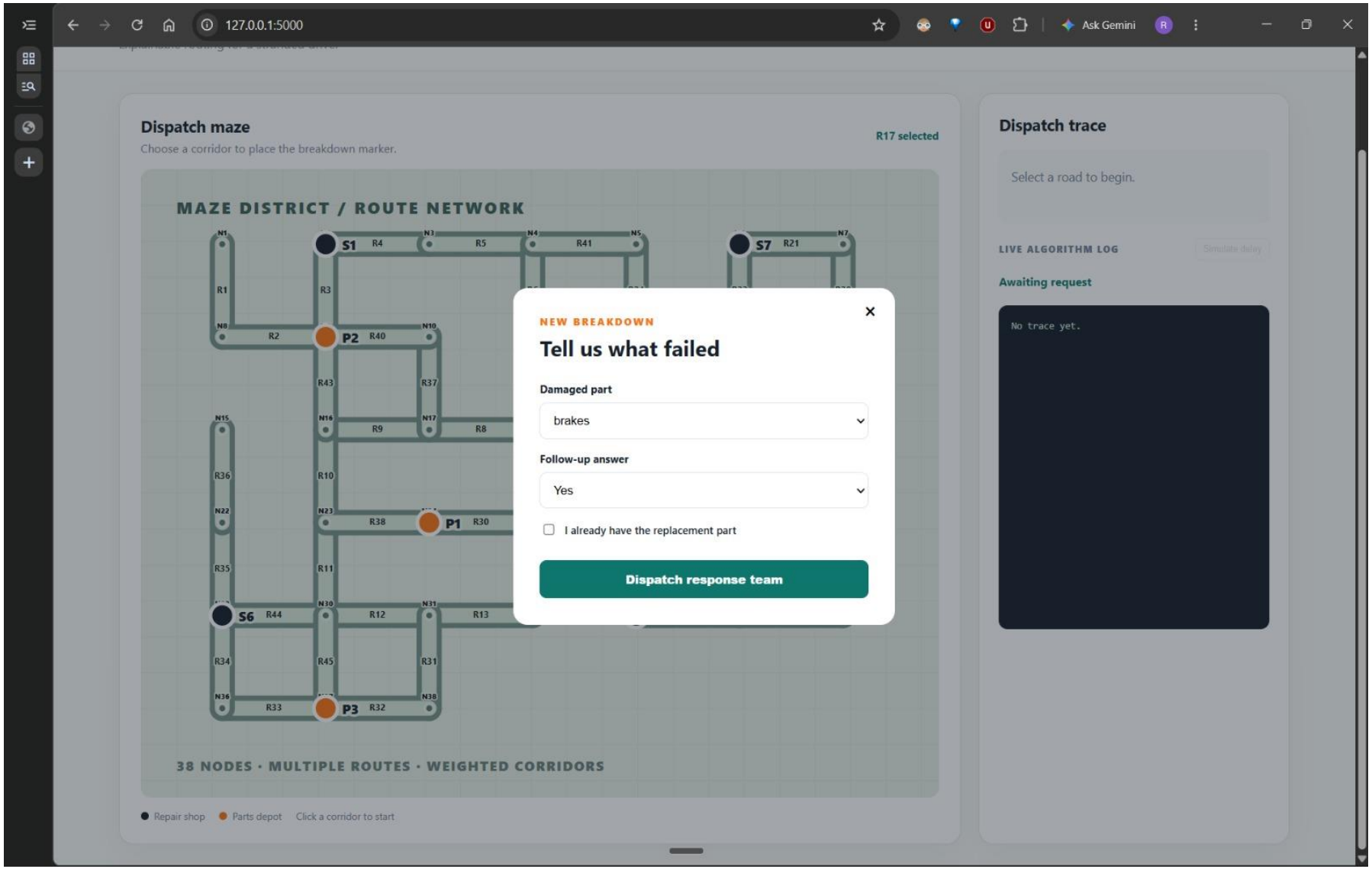
AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Results

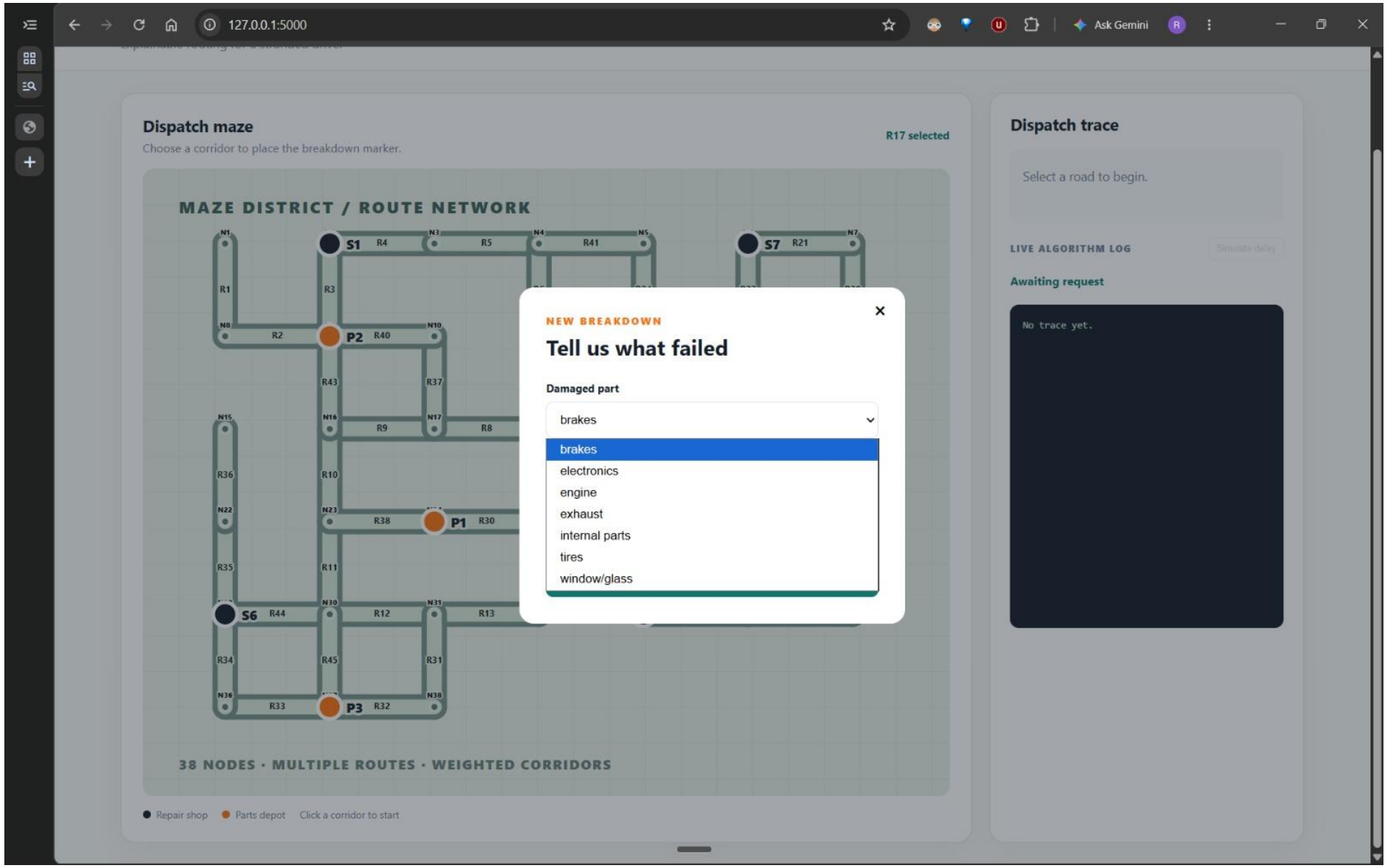


AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

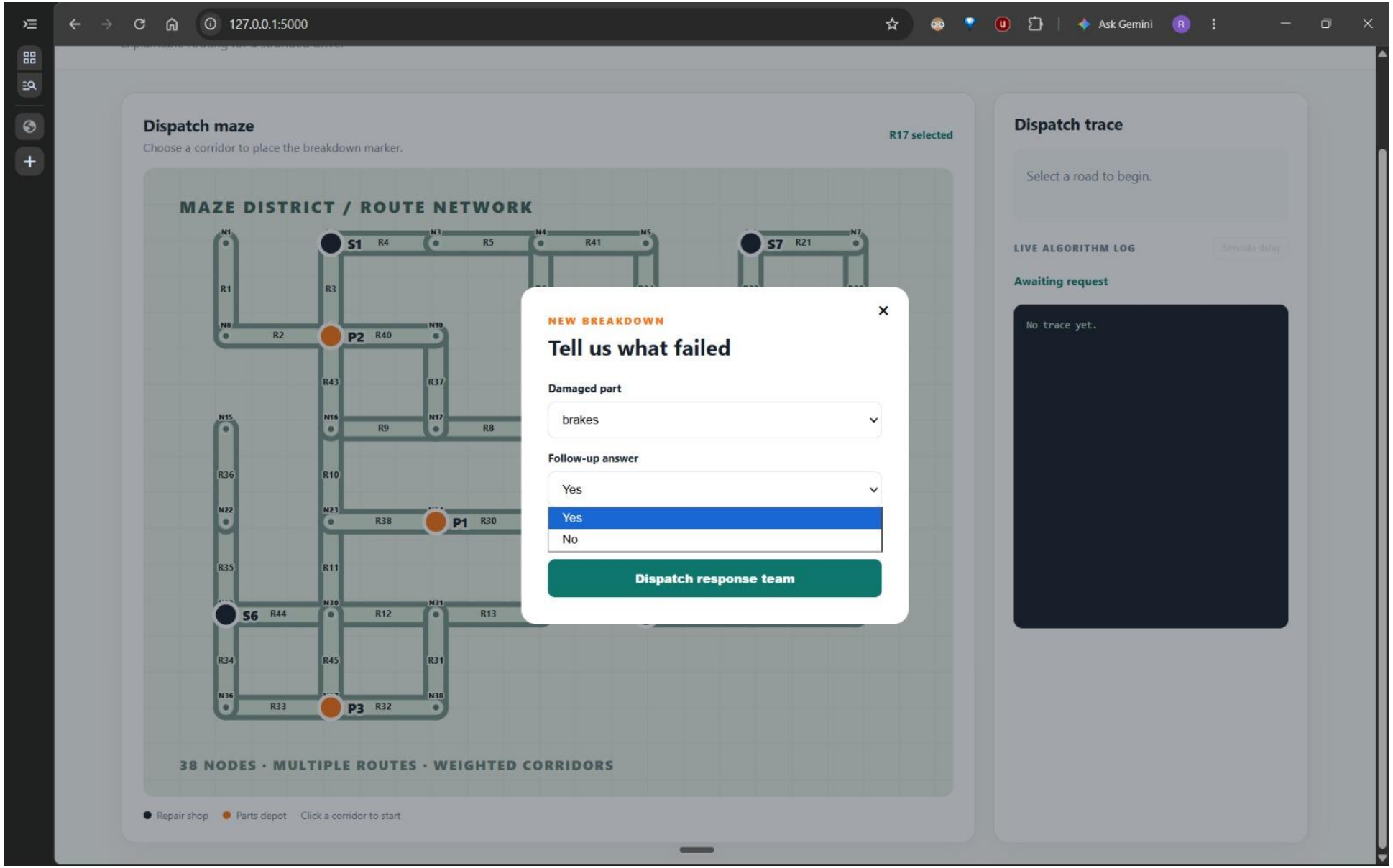
Results


127.0.0.1:5000
Ask Gemini
Dispatch maze
Choose a corridor to place the breakdown marker.
R17 selected
MAZE DISTRICT / ROUTE NETWORK
S1
S7
P2
P1
P3
S6
NEW BREAKDOWN
Tell us what failed
Damaged part
brakes
Follow-up answer
Yes
I already have the replacement part
Dispatch response team
38 NODES · MULTIPLE ROUTES · WEIGHTED CORRIDORS
Repair shop
Parts depot
Click a corridor to start
Dispatch trace
Select a road to begin.
LIVE ALGORITHM LOG
Awaiting request
No trace yet.

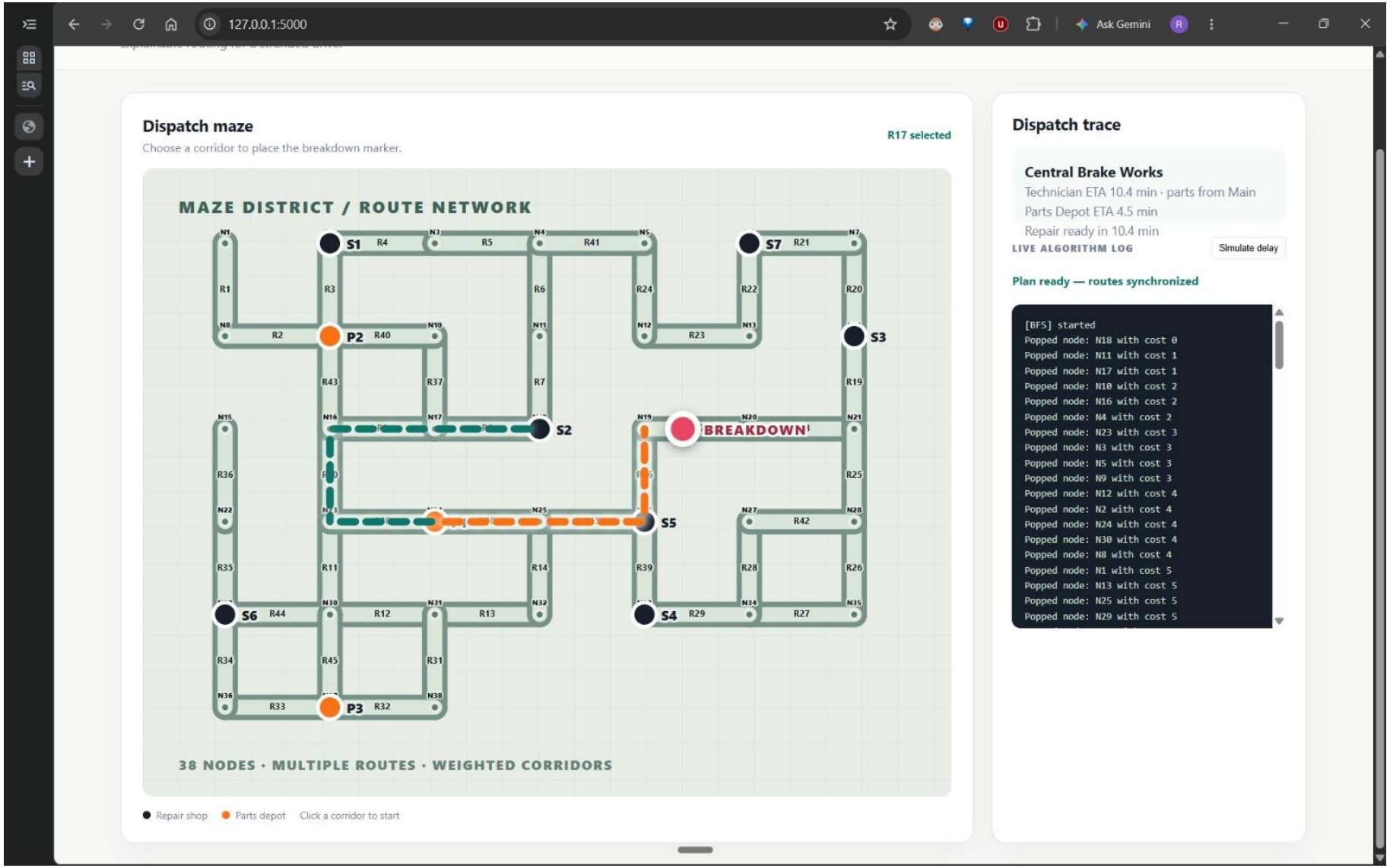
Results


127.0.0.1:5000
Ask Gemini
Dispatch maze
Choose a corridor to place the breakdown marker.
R17 selected
MAZE DISTRICT / ROUTE NETWORK
S1
S7
P1
P2
P3
S6
NEW BREAKDOWN
Tell us what failed
Damaged part
brakes
brakes
electronics
engine
exhaust
internal parts
tires
window/glass
38 NODES · MULTIPLE ROUTES · WEIGHTED CORRIDORS
Repair shop
Parts depot
Click a corridor to start
Dispatch trace
Select a road to begin.
LIVE ALGORITHM LOG
Awaiting request
No trace yet.

Results


127.0.0.1:5000
Ask Gemini
Dispatch maze
Choose a corridor to place the breakdown marker.
R17 selected
MAZE DISTRICT / ROUTE NETWORK
S1
S7
P1
P2
P3
S6
NEW BREAKDOWN
Tell us what failed
Damaged part
brakes
Follow-up answer
Yes
Yes
No
Dispatch response team
38 NODES · MULTIPLE ROUTES · WEIGHTED CORRIDORS
Repair shop
Parts depot
Click a corridor to start
Dispatch trace
Select a road to begin.
LIVE ALGORITHM LOG
Awaiting request
No trace yet.

Results


127.0.0.1:5000
Ask Gemini
Dispatch maze
Choose a corridor to place the breakdown marker.
R17 selected
MAZE DISTRICT / ROUTE NETWORK
S1
S2
S3
S4
S5
S6
S7
P2
P3
BREAKDOWN
38 NODES · MULTIPLE ROUTES · WEIGHTED CORRIDORS
Repair shop
Parts depot
Click a corridor to start
Dispatch trace
Central Brake Works
Technician ETA 10.4 min · parts from Main
Parts Depot ETA 4.5 min
Repair ready in 10.4 min
LIVE ALGORITHM LOG
Simulate delay
Plan ready — routes synchronized
[BFS] started
Popped node: N18 with cost 0
Popped node: N11 with cost 1
Popped node: N17 with cost 1
Popped node: N10 with cost 2
Popped node: N16 with cost 2
Popped node: N4 with cost 2
Popped node: N23 with cost 3
Popped node: N3 with cost 3
Popped node: N5 with cost 3
Popped node: N9 with cost 3
Popped node: N12 with cost 4
Popped node: N2 with cost 4
Popped node: N24 with cost 4
Popped node: N30 with cost 4
Popped node: N8 with cost 4
Popped node: N1 with cost 5
Popped node: N13 with cost 5
Popped node: N25 with cost 5
Popped node: N29 with cost 5



AMRITA
VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Thank you